TechPost ELEVATED

DC – AI – Routing – Switching – Security – Wireless – Automation

Multi-Node High Availability Basics

An HPE Juniper Networking Techpost

Steven Jacques

2024-12-20

HPE

Contents

1 Multi-Node High Availability Basics 3
1.1 Introduction . . . 3
1.2 Layer 2 versus Layer 3 HA . . . 3
1.3 Chassis Clustering – How it Works, and Its Limitations . . . 3
1.4 Multi-Node High Availability (MNHA) . . . 5
1.4.1 Inter-Chassis Link and Synchronisation . . . 6
1.4.2 Configuration Synchronisation . . . 8
1.4.3 Default Gateway Mode/SRG1+ . . . 8
1.4.4 Split Brain Detection and Prevention . . . 8
1.4.5 MNHA in Public Cloud . . . 9
1.5 MNHA Configuration Walkthrough . . . 9
1.6 Verification and Diagnostics . . . 14
1.7 Useful links . . . 16
1.8 Glossary . . . 16
1.9 Acknowledgments . . . 17

List of Figures

1 Chassis Clustering Topology 4
2 Topology example with MNHA 5
3 Inter-Chassis topology 7
4 Simple lab MNHA setup 10

1 Multi-Node High Availability Basics

Steven Jacques - 12/20/2024

In this post, we'll take a technical dive into Multi-Node High Availability (MNHA) on Juniper's SRX platforms – a flexible approach to providing redundancy on stateful network security devices.

1.1 Introduction

High Availability (HA) is a very commonly deployed function on NGFW platforms. Since one of the main tasks of the NGFW is to not only track but to *maintain* state (to secure connections and general network traffic), it is often vital that any redundancy/HA scheme used provides state synchronization between the member nodes of a redundant cluster. As a result, those nodes contain the same database of Run-Time Objects (RTOs) such as firewall sessions, IPSec Security Associations, and more.

This RTO synchronization facilitates failover between cluster members while keeping existing sessions alive. Consider a TCP session that is being used to transfer a file through a firewall cluster – if the primary cluster member fails without session synchronization, standard behaviour would be that state checking on the secondary unit would cause the session to be dropped.

HA architectures for NGFWs have often revolved around "Layer 2" or "VRRP-like" HA – where a virtual IP is shared and floats between the cluster members depending on current health state. However, this method has inherent limitations – so, in this post, we'll dive into Juniper's Multi-Node High Availability (MNHA) for SRX platforms, examine how it differs from traditional L2 HA, and look at how it works technically.

1.2 Layer 2 versus Layer 3 HA

Firstly, let's clarify some terminology with a touch of history. Chassis Clustering is the Juniper/SRX name for the Layer 2 method of HA (described next) and was introduced on the original SRX platforms, all the way back in 2008. This is supported and at the time of writing (November 2024) is still the predominant clustering method used in production on the SRX.

Multi-Node High Availability (MNHA) is the equivalent name for the layer 3 variant of HA. This was introduced roughly 3-4 years ago (range is given since functionality was rolled out over a number of releases). Here's a key point: MNHA has additional benefits while removing some limitations of chassis clustering. In most cases, we'd recommend anyone deploying new SRXs examine MNHA to see if it is a better fit than chassis clustering.

Let's look at the details.

1.3 Chassis Clustering – How it Works, and Its Limitations

Chassis clustering works by taking two SRX nodes and merging them to make a single logical chassis. We then configure Redundant Ethernet (Reth) interfaces – these are aggregate interface-like constructs that consist of physical links (or vNICs, in the case of the vSRX) from each of the two cluster nodes.

Any IP address assigned to a Reth interface is then a virtual IP, capable of existing on either node depending on which device is master of the Redundancy Group (RG) into which the Reth is placed. The master will now

respond to ARPs for the virtual IP; the backup node will not. At failover time, the secondary system will ARP gratuitously as it becomes master, to allow the layer 2 network to adjust the switch table entry for the virtual MAC. The concept can be seen below:

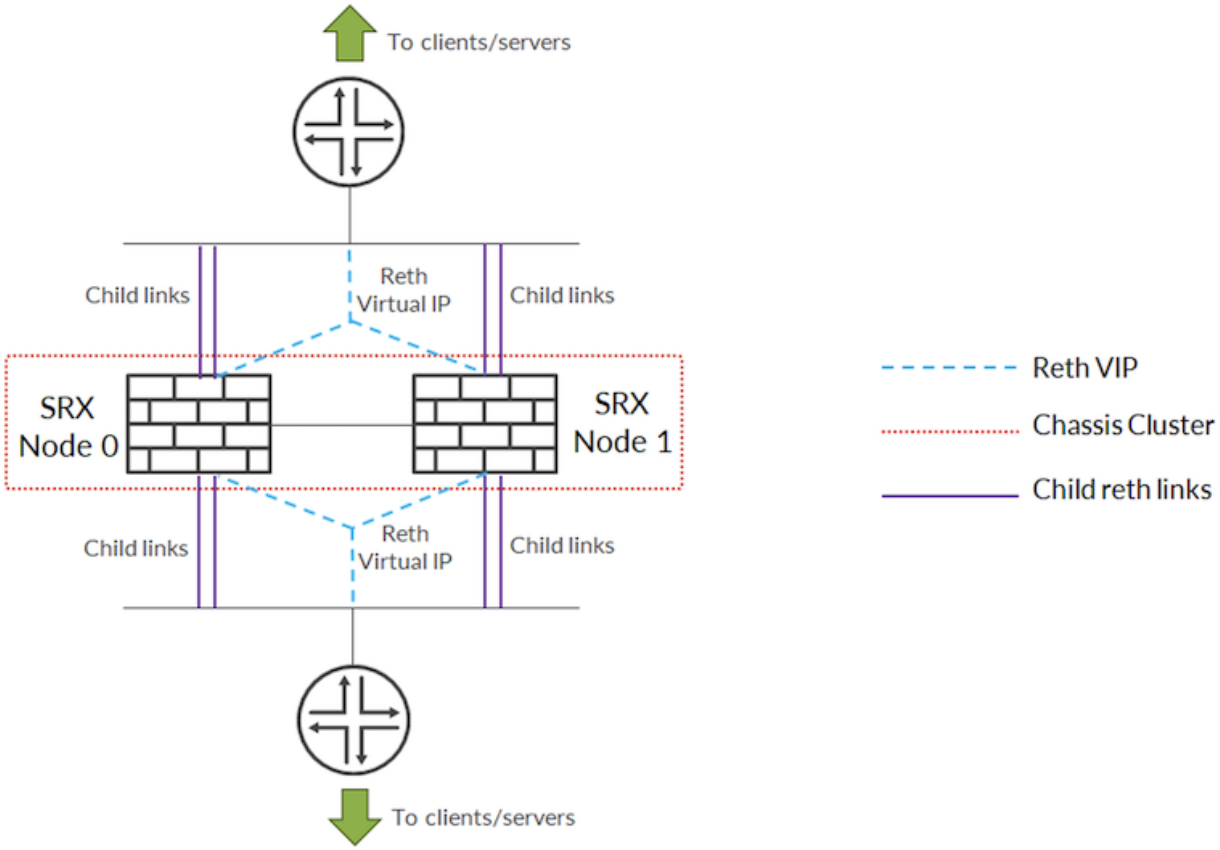


Figure 1: Chassis Clustering Topology

So far, so good. But using redundancy in this way has a couple of significant drawbacks:

- The cluster uses Virtual IPs, in most cases with Virtual MAC addresses underneath, which means that each member must be in the same layer 2 domain. This greatly reduces the ability to provision a 'geo-redundant' cluster.
- Chassis clustering uses Active/Passive control planes by design – and non-stop routing for route and peer synchronization is not supported on SRX platforms. This can cause problems in dynamic routing environments where the complete failure of the primary node means adjacencies/peerings must be re-established. Graceful Restart (GRES) can help here but isn't a silver bullet and longer failover times can be the result. BFD does not help here to speed up failover, unlike with MNHA.

Geo-redundancy for NGFW clusters is a design that is often desirable (and is becoming increasingly so) as architectures change, become more distributed, and so on. Consider PoP, or aggregation sites, or Datacenters, where the use of a cluster spanning the locations may be both cost-effective and technically superior. L2 HA means layer 2 stretches between these sites – and even if such infrastructure is available, other problems (such as the A/P control plane problem described above) still exist.

The solution? Layer 3 HA, or MNHA.

1.4 Multi-Node High Availability (MNHA)

MNHA has been available on SRX platforms for some time, but generally, 21.4 is considered the first release with full support. Note that various new features and enhancements have come with recent Junos versions – such as asymmetric flow support in 23.4. It's a good idea to check Pathfinder for details on which MNHA features are in which release if you are planning to deploy or migrate to MNHA.

So how does it work? The concept: from a network perspective, each SRX node in the cluster is completely standalone. So, in most cases, there are no interface VIPs or underlying VMACs as per chassis cluster – the cluster members have their own IP addresses, BGP peerings, routing policies, and so on. Note that for sessions to sync, the same physical and logical forwarding interfaces must be defined on the two nodes. That's to say, for example, you can't have your trust interface as ge-0/0/0 on one node and ge-0/0/2 on the other.

They can sit within completely separate network paths, and they present themselves to the network as independent nodes. MNHA does not have the concept of default configuration synchronization either – although selective synchronization is desirable and likely (more on that later). Finally, with MNHA, the nodes don't form a single virtual chassis.

Let's take the simple network diagram from the previous section and see how MNHA would fit:

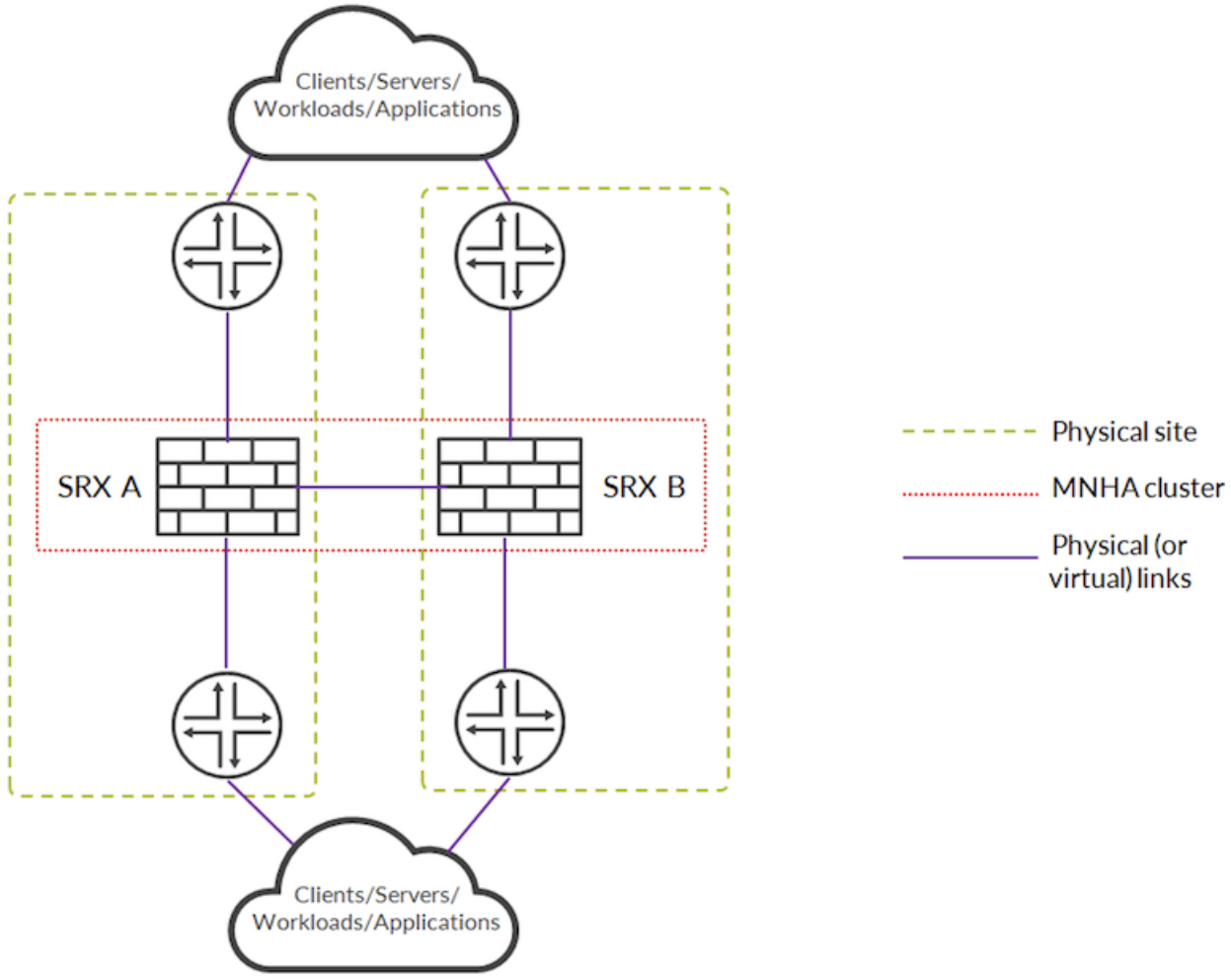


Figure 2: Topology example with MNHA

It's obviously not mandatory that the two MNHA cluster members are in different sites – they can just as

easily be in the same rack. However, since MNHA enables geo-redundancy, we'll draw it this way. From a geographical dispersion perspective, the only requirement is that the RTT between the two nodes must be less than 100ms – allowing significant distances between them.

MNHA means that the 'master' of the cluster is now determined by routing policy and network paths. In the default traffic forwarding group (known as a Services Redundancy Group or SRG, which is broadly similar to a redundancy group in chassis clustering) there is actually no concept of master and backup. Both nodes are "ready to go" at all times and will forward traffic according to policy and routing table if they receive any traffic – there is no requirement to switch ownership of the group, and no gratuitous ARP. This 'default' forwarding group is called SRG0.

One quick note on asymmetric routing. Although it was stated above that both nodes are hot, stateful firewalls and asymmetry don't play too nicely together. It's highly preferred to ensure that flows traverse the same node symmetrically. The SRXs DO support asymmetric flows, as of Junos 23.4 – but it is sub-optimal. We'll talk more about this in a subsequent blog.

The preferred or 'master' box is determined by any network routing policy attribute you like. In the diagram above, if we have BGP connections from each SRX cluster node to adjacent routers, we can control the preferred path with local-preference, MED, AS-path manipulation – routing policy is separate from MNHA functionality and of course, Junos has extremely rich routing capabilities. In the config walkthrough section later, we'll clarify this with a simple example.

An additional advantage of not having to switch ownership of a traffic group is potentially faster failover times. BFD over some routing protocol is the best way to achieve this, and for MNHA the SRX supports timing of 3 x 100ms (note that this is in distributed BFD mode – see here for more details on BFD on Junos). This means that on failure, failover time would be a maximum of 300ms. Remember that each device has its own independent control plane, so BGP peerings (for example) are up and advertising routes from each cluster node. This negates the problem we described earlier in the chassis cluster use case, where lack of NSR and reliance on GRES could mean longer failover times in dynamic routing environments.

Note that one thing that IS the same as a chassis cluster is Run-Time Object (RTO) synchronization. This includes firewall/NAT sessions, IPSec security associations, and more. This facilitates stateful failover when the network paths switch – i.e. existing flows in flight continue without interruption. Synchronization is done over the Inter-Chassis Link or ICL.

1.4.1 Inter-Chassis Link and Synchronisation

The MNHA diagram above depicts a direct connection between the cluster nodes, implying a dedicated and maybe even a directly connected back-to-back link. Neither is true. The link that the MNHA nodes use to communicate (for clustering information and RTO/configuration synchronization) is called the ICL and is a *logical* link. A typical configuration is an IP address on a loopback unit on each respective cluster node. Cluster communication is then done over whichever link/path should be used according to the routing table. Communication is at layer 3, so the loopback IPs can be in entirely different subnets.

This means that in-band forwarding interfaces are perfectly fine to use. Node-to-node cluster traffic is low – usually a maximum of low single-digit Gbps, even with high rates of RTO sync. Dedicated links are also fine but note that these do NOT use the HA ports available on some physical SRX platforms – those are for chassis clustering.

The most common is for one or two dedicated links to be used, with the in-band interfaces as backup. We end up with this scenario:

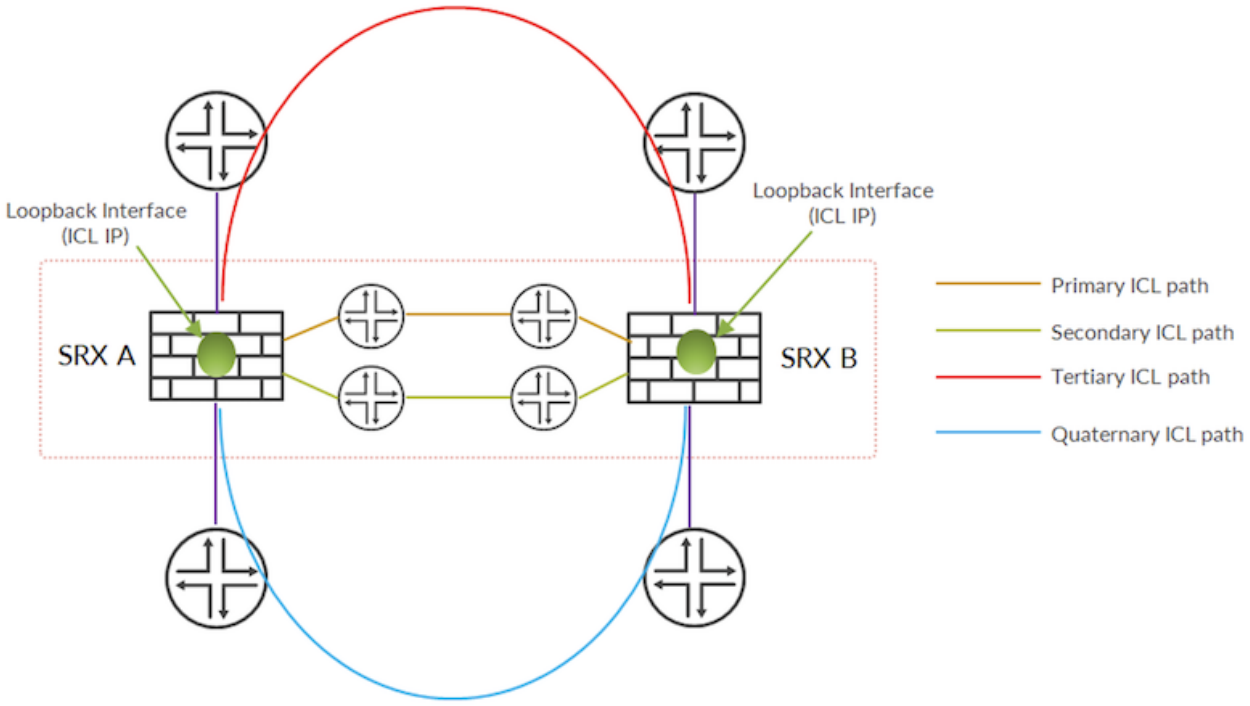


Figure 3: Inter-Chassis topology

Then, it's simply a case of configuring routing to prefer particular paths. If one path is unavailable, ICL communication will fall back to the next preferred one.

So what of synchronization? Run-Time Objects such as firewall sessions synchronize between members as they did with chassis clustering. But there's one difference – the sessions aren't marked as primary or secondary as they are with the chassis cluster, which reflects the fact that there is no Active/Passive formal relationship within the cluster:

```
Session ID: 2002, Policy name: internal/5, HA State: Active, Timeout: 300, Session State: Valid
  In: 10.0.30.10/58558 --> 10.0.35.10/80;tcp, Conn Tag: 0x0, If: ge-0/0/0.100, Pkts: 424, Bytes:
      22182, HA Wing State: Active,
  Out: 10.0.35.10/80 --> 10.0.30.10/58558;tcp, Conn Tag: 0x0, If: ge-0/0/0.101, Pkts: 14556,
      Bytes: 21831112, HA Wing State: Active,
```

and

```
Session ID: 1949, Policy name: internal/5, HA State: Warm, Timeout: 2342, Session State: Valid
  In: 10.0.30.10/58558 --> 10.0.35.10/80;tcp, Conn Tag: 0x0, If: ge-0/0/0.100, Pkts: 0, Bytes:
      0, HA Wing State: Warm,
  Out: 10.0.35.10/80 --> 10.0.30.10/58558;tcp, Conn Tag: 0x0, If: ge-0/0/0.101, Pkts: 0, Bytes:
      0, HA Wing State: Warm,
```

Instead, they are marked as 'Active' and 'Warm', and 'Active' does not reflect cluster state – it's the status of the session on that node. So, if traffic flipped to the secondary node, as soon as any packets matched the session on the second node, it would adjust the marker from 'Warm' to 'Active', and notify the other node, which would adjust from 'Active' to 'Warm'. As mentioned, either node is ready to forward traffic for this session at any time.

1.4.2 Configuration Synchronisation

But how about configuration synchronization? Network parameters obviously cannot sync – we've stated that the cluster nodes in MNHA have unique IP addresses, peerings, and so on. However, parts of the configuration should or must sync – such as security policies, and IPSec tunnel configurations – or RTOs won't be installed correctly on the peer node.

In MNHA, when you're using the CLI to manage the SRXs, configuration stanzas are synched by Junos group definitions (meaning the administrator has control of what needs to be synchronized). We'll cover the use of groups in the configuration walkthrough section.

Security Director and Security Director Cloud – Juniper's central management platform – supports MNHA, so if you are configuring policies that way, synchronization will be handled automatically.

1.4.3 Default Gateway Mode/SRG1+

So far, we've discussed the fact that MNHA leverages two cluster members at different layer 3 locations in the network, and not a shared VIP/vMAC on a common layer 2 domain. However, MNHA also supports layer 2 mode. If we want to use a VIP and vMAC that moves between the two nodes in MNHA dependent on failure conditions, we can. It's known as default gateway mode and essentially operates in a similar way to chassis clustering.

Routing mode (the "No VIP" or layer 3 mode) can be done in either SRG0 or SRG1+. SRG0 does not have an Active/Passive state; as mentioned previously both nodes are Active and ready to forward traffic at any time (i.e. one node does not have to transition from Passive to Active on failover). However, SRG1+ typically uses virtual IPs – either IKE/IPSec termination points (normally on loopback interfaces) or standard virtual IPs on external interfaces. If you are terminating IPSec traffic on an MNHA cluster and want the tunnel to failover between the two SRXs, you MUST use SRG1 or higher, not SRG0.

Since SRG1+ has a VIP for IPSec tunnels or a default gateway, each SRG of this type DOES operate as Active/Passive. We can see the status of this in diagnostics (again, see later) and once again this behaves very similarly to a chassis cluster redundancy group.

The SRXs can also handle traffic in Active/Active mode using multiple SRGs. As an example, if we have SRG1 and SRG2 configured, SRG1 could be used for IPSec termination with a loopback IP which is normally active on node A, and SRG2 could be a default gateway for clients with a VIP which is normally active on node B.

The support for layer 2/default gateway mode helps to smooth the transition to MNHA if you are using an SRX in a chassis cluster. You can keep the same VIP configuration and effectively mirror the chassis cluster deployment but still take advantage of MNHA benefits (such as Active/Active control planes, which is always true).

1.4.4 Split Brain Detection and Prevention

A split brain in a network is… bad. Alright, it's fair to say that it's not as extreme as Ghostbusters-what-happens-if-we-cross-the-streams bad, but it's still undesirable. Split brain in a stateful firewall context usually means both nodes in a cluster are responding to ARPs for the same VIP – meaning the L2 infrastructure may have two ports associated with the same vMAC, and you get entirely unpredictable forwarding across the nodes. Downtime normally ensues.

The good news is that we have already said there's no Active/Passive VIP when using SRG0 with MNHA. So if the two nodes suddenly can't see each other, the only impact is that there is no state synchronization, which in the worst case would lead to some sessions needing to be re-established IF the network path switches to the secondary node while split brain is occurring. There'd still be a single preferred path per routing policy because that's not dependent on being able to see and communicate with the other MNHA cluster node. Generally, things would continue happily – split-brain isn't really a concept in an SRG0-only deployment.

However, if using SRG1+, here is an Active/Passive state per SRG. In this case – especially since we can use interface VIPs in MNHA if we like ('Default Gateway' mode) – split brain is possible. Watch out for another future blog talking more about SRG1+ and diving deep on probing and other mechanisms, but for now:

- Split brain probing can be ICMP or BFD based;
- When the ICL goes down, a probe is sent to a network device from both nodes, using the VIP as the source;
- Since the network has preferred paths, it will respond to both probes via the same SRX. Therefore, if the current backup node sends a probe and does not receive a response, it assumes that the response was sent to the still-active preferred path, and it remains backup.

1.4.5 MNHA in Public Cloud

Stateful redundancy in public clouds such as AWS and Azure has, historically, been an issue due to one particular problem – the public cloud tenant typically has no control or visibility into layer 2 functionality and as a rule, any links are provisioned as pure L3. So, explicitly provisioning a layer 2 stretch across a pair of virtual SRXs is not possible – meaning the chassis cluster model generally cannot be made to work. MNHA is pure layer 3, so we CAN use it in public cloud – it requires a little extra secret sauce since access is usually via elastic IPs and the vSRX has to notify the AWS infra that the elastic IP binding has moved to the second vSRX by calling into the AWS SDK API. The general configuration and concept are the same though. We won't cover MNHA on public cloud in this blog, but see here for some details for AWS.

1.5 MNHA Configuration Walkthrough

Ok, let's take a look at how to get up and running with MNHA. For this, we'll be using the simple lab shown below:

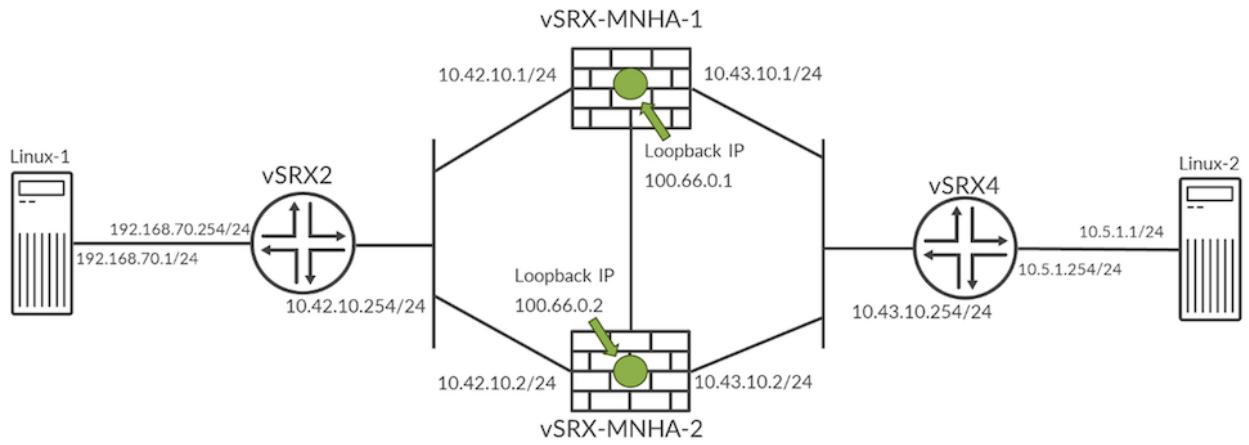


Figure 4: Simple lab MNHA setup

We won't cover basic Junos setup such as assigning IP addresses, and nor will we cover the security portions in detail. From a firewall zone/policy perspective, we're just using a pair of zones on each vSRX with a default permit policy.

Let's start by looking at the clustering configuration. In the case where we use SRG0 only, this is very straightforward. This blog only covers the use of SRG0, but note that the L3 mode functionality covered below can also be done in SRG1+ - and those higher SRG numbers have additional functionality, such as Active/Backup signal route, activeness priority, preemption, VIPs, and more. Therefore, it may be beneficial to not use SRG0 at all, but instead to use the configuration below in SRG1 or above. We'll dive into that in our second blog. For now, note that everything below will work perfectly well to get your MNHA cluster up and running:

```
root@vSRX-MNHA-1# show chassis high-availability
local-id {
    1;
    local-ip 100.66.0.1;
}
peer-id 2 {
    peer-ip 100.66.0.2;
    interface lo0.0;
    routing-instance icl;
    liveness-detection {
        minimum-interval 1000;
        multiplier 3;
    }
}
services-redundancy-group 0 {
    peer-id {
        2;
    }
}
```

MNHA configuration uses the 'chassis high-availability' hierarchy (whereas L2 HA/chassis clustering unsurprisingly uses the 'chassis cluster' commands). The config seen above is everything you need to get the two MNHA nodes into a cluster and forwarding traffic. Of course, outside of the MNHA configuration, you'd probably need to set up your routing peerings, routing policy, security policies, and so on. Those, however, are disaggregated and in no way related to the MNHA config.

The SRG0 cluster config is fairly self-explanatory. The local-id is a numerical value per-node. The local IP is the address used for the Inter-Chassis Link (ICL) which is used for heartbeats, Run-Time Object synchronization, and so on. The peer-id config defines the attributes of the other node in the cluster and specifies which interface to use to contact it (lo0.0 – note that we must explicitly configure this address on lo0.0, the chassis high-availability commands won't do it for us:

```
root@vSRX-MNHA-1# show interfaces lo0
unit 0 {
   description ICL;
   family inet {
      address 100.66.0.1/32;
   }
}
```

Finally, continuing through the chassis high-availability config shown previously, we add liveness-detection to verify the health of the ICL (remember that we stated that the ICL is logical and can switch between any available path) and we define SRG0 and the peer-id value. We're done! Almost…note that you must make sure that the interfaces over which the logical ICL will run have been assigned to a zone, and that zone permits SSH (for peer synchronization), High-Availability, and (if using ICL encryption) IKE:

```
root@vSRX-MNHA-1# show security zones
security-zone icl {
   host-inbound-traffic {
      system-services {
         ssh;
         ike;
         high-availability;
      }
      protocols {
         all;
      }
   }
   interfaces {
      ge-0/0/1.0;
      lo0.0;
   }
}
```

This above configuration shows SRG0 without ICL encryption. However, ICL encryption is recommended as best practice, so we're also going to enable this:

```
root@vSRX-MNHA-2# show chassis high-availability | match vpn | display set
set chassis high-availability peer-id 1 vpn-profile icl

root@vSRX-MNHA-2# show groups mnha-sync security ike | display set
set groups mnha-sync security ike proposal ike-prop authentication-method pre-shared-keys
set groups mnha-sync security ike proposal ike-prop dh-group group20
set groups mnha-sync security ike proposal ike-prop encryption-algorithm aes-256-gcm
set groups mnha-sync security ike proposal ike-prop lifetime-seconds 28800
set groups mnha-sync security ike policy icl proposals ike-prop
set groups mnha-sync security ike policy icl pre-shared-key ascii-text "$9$fQ390BEevLApvLxNY25QF
    "
set groups mnha-sync security ike gateway icl ike-policy icl
set groups mnha-sync security ike gateway icl version v2-only

root@vSRX-MNHA-2# show groups mnha-sync security ipsec | display set
set groups mnha-sync security ipsec proposal ipsec-prop encryption-algorithm aes-256-gcm
set groups mnha-sync security ipsec proposal ipsec-prop lifetime-seconds 3600
set groups mnha-sync security ipsec policy ipsec-policy perfect-forward-secrecy keys group20
set groups mnha-sync security ipsec policy ipsec-policy proposals ipsec-prop
set groups mnha-sync security ipsec policy mnha proposal-set standard
set groups mnha-sync security ipsec vpn icl ha-link-encryption
set groups mnha-sync security ipsec vpn icl ike gateway icl
set groups mnha-sync security ipsec vpn icl ike ipsec-policy ipsec-policy
```

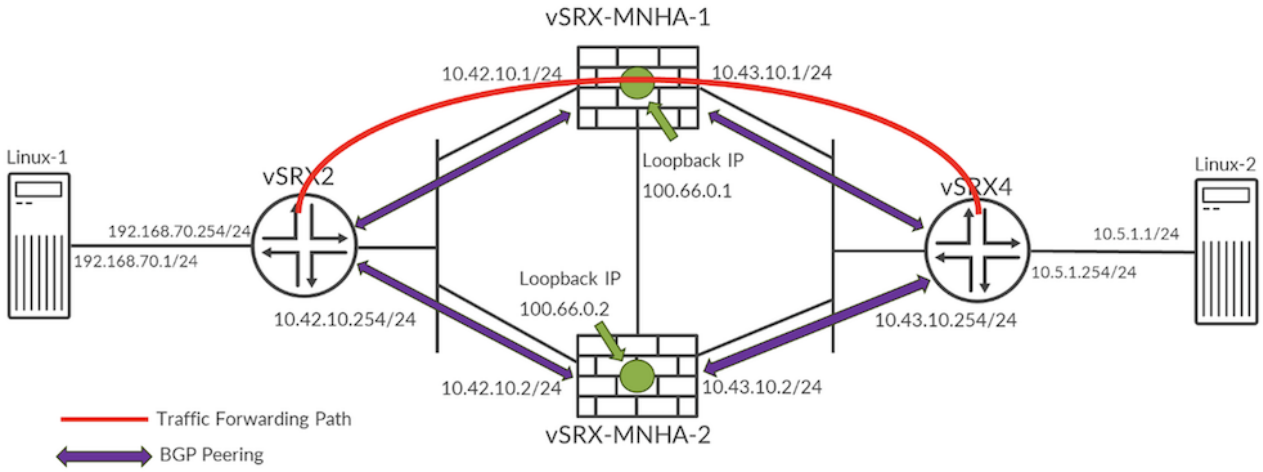
Note that while the encryption is IPSec, the configuration is different from a standard site-to-site VPN on SRX. We don't define an IP address on the IKE gateway, for starters. And you must set the 'ha-link-encryption' setting on the IPSec VPN. In addition, various options for IKE and IPSec configuration aren't supported with link encryption – you'll get commit errors if you try to enable some other settings.

Note that MNHA link encryption is only possible with the IKED IKE/IPSec daemon – not the older KMD. Some Junos platforms have KMD as the default package, and you'll need to install IKED yourself. To check whether your platform already has IKED, you can look here – if it's listed as an optional package, you will need to install it. Use the following command to do so:

```
request system software add optional://junos-ike.tgz
```

There are a couple of other configuration elements you're likely to need. Firstly, we need to add routing peers (BGP in this lab case) and routing policy. For this first example, we're going as simple as it gets. vSRX2 and vSRX4 have BGP peerings with vSRX-MNHA-1 and -2, and they each redistribute the connected subnets with the Linux test hosts (192.168.70.0/24 and 10.5.1.0/24) into BGP, which is then advertised onwards. We need one of the two vSRX MNHA nodes to be preferred, so we'll simply apply an import policy on BGP routes with a local-preference, meaning vSRX-MNHA-1 is preferred.

We end up with this:



Below is how the BGP and policy configuration looks on vSRX2. This is really basic Junos! Obviously, a production routing policy may be much more complex, but that's the beauty of MNHA – you can do whatever you want with the routing policy since it's not directly linked to MNHA functionality.

```
root@vSRX2# show policy-options
prefix-list export-connected {
    192.168.70.0/24;
}
policy-statement From_1 {
    from protocol bgp;
    then {
        local-preference 200;
    }
}
policy-statement From_2 {
    from protocol bgp;
    then {
        local-preference 150;
    }
}
policy-statement export-to-mnha {
    term 10 {
        from {
            protocol direct;
            prefix-list export-connected;
        }
        then accept;
    }
}

root@vSRX2# show protocols bgp
group vsrx-mnha-2 {
    export export-to-mnha;
    peer-as 65032;
    neighbor 10.42.10.2 {
        import From_2;
    }
}
group vsrx-mnha-1 {
    export export-to-mnha;
```

```
    peer-as 65031;
    neighbor 10.42.10.1 {
       import From_1;
    }
}
local-as 65030;
bfd-liveness-detection {
    minimum-interval 1000;
    multiplier 3;
}
```

The above gives us a preferred route on vSRX2 via vSRX-MNHA-1 (we repeat the same on vSRX4 in this test lab to keep the routing symmetrical):

```
root@vSRX2> show route

inet.0: 20 destinations, 21 routes (20 active, 0 holddown, 0 hidden)
+ = Active Route, - = Last Active, * = Both

10.5.1.0/24 *[BGP/170] 4d 04:51:25, localpref 200
                AS path: 65031 65033 I, validation-state: unverified
              > to 10.42.10.1 via ge-0/0/2.0
              [BGP/170] 4d 05:00:16, localpref 150
                AS path: 65032 65033 I, validation-state: unverified
              > to 10.42.10.2 via ge-0/0/2.0
```

Now, the last configuration piece, which is the security functionality on the MNHA nodes. So far, everything we've talked about has NOT been synchronized between the two nodes. They need their own unique IP addresses, BGP peerings, routing policies, and so on. However, when it comes to security functions, they will likely be the same on both nodes. Firewall policies should be consistent across the two MNHA cluster nodes, so in this case, we're going to configure them such that they're automatically synchronized.

To do this, we use Junos groups, and define a group with a specific condition:

```
root@vSRX-MNHA-1# show groups
mnha-sync {
    when {
       peers [ vSRX-MNHA-1 vSRX-MNHA-2 ];
    }
    security {
       ....
          }
```

We won't cover the functionality of groups in depth here. They're essentially a way to create common config elements and apply them in specific ways. If you haven't seen them before, check here for a good primer. In this case, per the above config, we create a group called 'mnha-sync' (call it whatever you like) and then we use the 'then…peers' condition. The effect this has is that anything configured under the group hierarchy (which can be any Junos config command) is applied to both 'vSRX-MNHA-1' and 'vSRX-MNHA-2'. vSRX-MNHA-1 is the local hostname in this case, so we also need to tell this SRX what and where 'vSRX-MNHA-2' is:

```
root@vSRX-MNHA-1# show system | display set
set system host-name vSRX-MNHA-1
set system services netconf ssh
set system commit peers vSRX-MNHA-2 user root
set system commit peers vSRX-MNHA-2 authentication "$9$Gxjkmz39O1hfT1hSr8LGDi"
set system static-host-mapping vSRX-MNHA-2 inet 172.30.192.156
set security ssh-known-hosts host vsrx-mnha-2,172.30.192.156 ecdsa-sha2-nistp384-key AAAA---snip
    ---ioCmqQ==
```

With the above commands, we give the node the ability to take the configuration commands entered under the mnha-sync group and push them to the other node, using the IP address and credentials defined. We repeat this configuration on the vSRX-MNHA-2, but obviously with the .1 IP address and flipped hostnames. This means that we can enter configuration commands into the group on either node and those commands will synchronize.

There are a couple of things to note with the above config synchronization commands. Firstly, you can synchronize across any interface you choose – normally either the ICL or fxp0, the out-of-band management interface. In the case above, the static-host-mapping is the peers' fxp0 address, so that's what will be used. Secondly, the final command is the SSH key fingerprint for the MNHA cluster peer. This is needed for the config synchronization to work. The command you actually enter in config mode is as follows – once retrieved, the SRX converts the command to the above automatically:

```
root@vSRX-MNHA-1# set security ssh-known-hosts fetch-from-server vSRX-MNHA-2
The authenticity of host 'vsrx-mnha-2 (172.30.192.156)' can't be established.
ECDSA key fingerprint is SHA256:xQe1i9wKprwrv7ZV1P2Vwx45D9c0Bwc5Qr1Lv7yjHtE.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added 'vsrx-mnha-2,172.30.192.156' (ECDSA) to the list of known hosts.
```

You then configure commands in the group as normal commands. You're generally going to want things like security zones, security policies, IPSec tunnel definitions, and so on, in the group, so that the security configuration and hence posture are identical between the two devices. But it's all optional. Need one of the devices in the cluster to have different security policies for some reason? No problem – just create the policy outside of the group config, and it will not synchronize to the peer. MNHA cluster nodes only sync configuration based on the above group method. Just remember that sessions matching any non-synchronized policy you create would also themselves not be transferred to the other node as a run-time object.

Finally, we apply the group to the overall configuration, and add the peers-synchronize command, which will sync the configuration at commit time. Note that you can also use the 'peers-synchronize' attribute as an explicit command attribute when doing a commit (i.e., type 'commit peers-synchronize' in config mode). This may be a good option to manually verify synchronization at commit time.

```
set apply-groups mnha-sync
set system commit peers-synchronize
```

1.6 Verification and Diagnostics

With SRG0, we don't have an Active/Passive state, as we previously discussed. So, when it comes to ensuring that our configuration is working, there are really just two things to do. Firstly, we can check that the ICL is up and that the nodes are communicating:

```
root@vSRX-MNHA-1> show chassis high-availability information
Node failure codes:
   HW Hardware monitoring LB Loopback monitoring
   MB Mbuf monitoring SP SPU monitoring
   CS Cold Sync monitoring SU Software Upgrade

Node Status: ONLINE
Local-id: 1
Local-IP: 100.66.0.1
HA Peer Information:

   Peer Id: 2 IP address: 100.66.0.2 Interface: lo0.0
   Routing Instance: icl
   Encrypted: NO Conn State: UP
   Configured BFD Detection Time: 3 * 1000ms
   Cold Sync Status: COMPLETE

Services Redundancy Group: 0
      Current State: ONLINE
      Peer Information:
        Peer Id: 2
```

Above we can see that the connection state is up, meaning communication between the two MNHA nodes is working. Cold sync is complete (cold sync is the process of synchronizing run-time objects such as sessions

after some disconnect between the nodes). If we deliberately break the ICL, we can see the status change:

```
root@vSRX-MNHA-1> show chassis high-availability information
Node failure codes:
   HW Hardware monitoring LB Loopback monitoring
   MB Mbuf monitoring SP SPU monitoring
   CS Cold Sync monitoring SU Software Upgrade

Node Status: ONLINE
Local-id: 1
Local-IP: 100.66.0.1
HA Peer Information:

   Peer Id: 2 IP address: 100.66.0.2 Interface: lo0.0
   Routing Instance: icl
   Encrypted: NO Conn State: DOWN
   Configured BFD Detection Time: 3 * 1000ms
   Cold Sync Status: IN PROGRESS

Services Redundancy Group: 0
      Current State: ONLINE
      Peer Information:
       Peer Id: 2
```

Secondly, we can check sessions – firstly, that they synchronize to the other node, and also, that failover is stateful and the session continues to work. Let's start an SSH session from Linux-1 to Linux-2 (refer back to the lab diagram if needed):

```
root@vSRX-MNHA-1> show security flow session destination-port 22
Session ID: 20911, Policy name: default-policy-logical-system-00/2, HA State: Active, Timeout: 1790, Session State: Valid
  In: 192.168.70.1/60314 --> 10.5.1.1/22;tcp, Conn Tag: 0x0, If: ge-0/0/0.0, Pkts: 19, Bytes: 3813, HA Wing State: Active,
 Out: 10.5.1.1/22 --> 192.168.70.1/60314;tcp, Conn Tag: 0x0, If: ge-0/0/2.0, Pkts: 18, Bytes: 4442, HA Wing State: Active,
Total sessions: 1

root@vSRX-MNHA-2> show security flow session destination-port 22
Session ID: 23127, Policy name: default-policy-logical-system-00/2, HA State: Warm, Timeout: 14364, Session State: Valid
  In: 192.168.70.1/60314 --> 10.5.1.1/22;tcp, Conn Tag: 0x0, If: ge-0/0/0.0, Pkts: 0, Bytes: 0, HA Wing State: Warm,
 Out: 10.5.1.1/22 --> 192.168.70.1/60314;tcp, Conn Tag: 0x0, If: ge-0/0/2.0, Pkts: 0, Bytes: 0, HA Wing State: Warm,
Total sessions: 1
```

Now let's failover by forcing the routing to flip (not by changing any HA state on the SRX). We'll simply disable a forwarding interface on vSRX-MNHA-1. We see the status flip – and crucially, the SSH session continues to work, without any noticeable pause (due to the use of BFD with BGP, failover is fast – less than half a second is possible. BFD is not mandatory though):

```
root@vSRX-MNHA-1> show security flow session destination-port 22
Session ID: 20911, Policy name: default-policy-logical-system-00/2, HA State: Warm, Timeout: 1802, Session State: Valid
  In: 192.168.70.1/60314 --> 10.5.1.1/22;tcp, Conn Tag: 0x0, If: ge-0/0/0.0, Pkts: 32, Bytes: 4669, HA Wing State: Warm,
 Out: 10.5.1.1/22 --> 192.168.70.1/60314;tcp, Conn Tag: 0x0, If: ge-0/0/2.0, Pkts: 26, Bytes: 5418, HA Wing State: Warm,
Total sessions: 1

root@vSRX-MNHA-2> show security flow session destination-port 22
Session ID: 23127, Policy name: default-policy-logical-system-00/2, HA State: Active, Timeout: 1712, Session State: Valid
  In: 192.168.70.1/60314 --> 10.5.1.1/22;tcp, Conn Tag: 0x0, If: ge-0/0/0.0, Pkts: 31, Bytes: 1984, HA Wing State: Active,
 Out: 10.5.1.1/22 --> 192.168.70.1/60314;tcp, Conn Tag: 0x0, If: ge-0/0/2.0, Pkts: 21, Bytes: 2376, HA Wing State: Active,
Total sessions: 1
```

That's it! Our MNHA cluster is up and running and traffic fails over successfully and statefully.

There's a lot more nuance to MNHA – look out for a follow-up blog where we'll dive into SRG1+, advanced

monitoring, deeper routing policies matching to SRG1 state, and more. As a final point of interest before that, here's what the MNHA diagnostic output looks like when using SRG1+ - we can clearly see the Active/Passive relationship as well as a lot more detail than with SRG0!

```
root@vSRX-MNHA-1# run show chassis high-availability information
Services Redundancy Group: 0
      Current State: ONLINE
      Peer Information:
        Peer Id: 2
Services Redundancy Group: 1
      Deployment Type: ROUTING
      Status: ACTIVE
      Activeness Priority: 200
      Preemption: DISABLED
      Process Packet In Backup State: YES
      Control Plane State: READY
      System Integrity Check: N/A
      Failure Events: NONE
      Peer Information:
        Peer Id: 2
        Status : BACKUP
        Health Status: HEALTHY
        Failover Readiness: READY

root@vSRX-MNHA-2> show chassis high-availability information
Services Redundancy Group: 0
      Current State: ONLINE
      Peer Information:
        Peer Id: 1
Services Redundancy Group: 1
      Deployment Type: ROUTING
      Status: BACKUP
      Activeness Priority: 100
      Preemption: DISABLED
      Process Packet In Backup State: YES
      Control Plane State: READY
      System Integrity Check: COMPLETE
      Failure Events: NONE
      Peer Information:
        Peer Id: 1
        Status : ACTIVE
        Health Status: HEALTHY
        Failover Readiness: N/A
```

1.7 Useful links

- Multi-Node High Availability: public documentation https://www.juniper.net/documentation/us/en/software/junos/high-availability/topics/topic-map/mnha-introduction.html
- Pathfinder – feature support per-platform and other useful information https://apps.juniper.net/home/segment?segment=Security&subSegment=Security
- Bi-Directional Forwarding Detection (BFD) guide https://www.juniper.net/documentation/us/en/software/junos/high-availability/topics/topic-map/bfd.html
- MNHA in-service upgrade script https://github.com/Juniper/mnha-software-upgrade

1.8 Glossary

- API Application Programming Interface
- AS-Path Autonomous System Path
- BFD Bi-Directional Forwarding Detection
- BGP Border Gateway Protocol
- Gbps Gigabits per second
- GRES Graceful Restart

- HA High Availability
- ICL Inter-Chassis Link
- MED Multi-Exit Discriminator
- MNHA Multi-Node High Availability
- NAT Network Address Translation
- NGFW Next Generation Firewall
- NSR Non-Stop Routing
- RETH Redundant Ethernet (interface)
- RTO Run-Time Object
- RTT Round-Trip Time
- SDK Software Development Kit
- SRG Services Redundancy Group
- VIP Virtual IP (address)
- VMAC Virtual Media Access Control (Virtual MAC address)
- VNIC Virtual Network Interface Card

1.9 Acknowledgments

Karel Hendrych for sanity checking, corrections, and more. And Juniper dev teams for building the MNHA feature in the first place!



DC – AI – Routing – Switching – Security – Wireless – Automation

© Copyright 2026 Hewlett Packard Enterprise Development LP. The information contained herein is subject to change without notice. The only warranties for Hewlett Packard Enterprise products and services are set forth in the express warranty statements accompanying such products and services. Nothing herein should be construed as constituting an additional warranty.
Hewlett Packard Enterprise shall not be liable for technical or editorial errors or omissions contained herein.
Trademark acknowledgments, if needed. All third-party marks are the property of their respective owners.

HEWLETT PACKARD ENTERPRISE

HPE.com



